

Path-Aware Governance for High-Volume AI-Agent Workflows

AMS 560 / CSE 542 Project Proposal

Research Question. How effectively can path-aware governance improve policy enforcement in high-volume AI-agent workflows compared with no governance and lightweight rule-based governance, while maintaining acceptable system performance?

1 Problem Statement

As AI agents become increasingly capable and widely deployed, organizations face growing concerns over security, data privacy, and the unintended actions that autonomous agents may take during execution. These risks have increased the need for effective runtime governance, mechanisms that can monitor and control agent behavior as actions occur.

Existing governance frameworks and industry guidelines provide important principles for responsible AI use, but many practical enforcement mechanisms still rely on relatively static controls, such as role-based permissions, predefined access rules, or isolated policy checks. Such controls may be insufficient for complex, multi-step agent workflows, where the appropriateness of an action depends not only on the current request but also on the sequence of prior actions and the context through which the agent reached that point.

This project proposes a path-aware governance framework for high-volume AI-agent workflows that incorporates an agent’s execution path into runtime governance decisions. We implement and demonstrate the framework under high-volume workloads and compare three configurations: no governance, lightweight rule-based governance, and path-aware governance. The evaluation examines both governance effectiveness and system-level trade-offs using metrics such as policy-violation detection, false-positive and false-negative rates, latency, throughput, and resource utilization. The goal is to determine whether path-aware governance can provide more effective runtime control for complex agent workflows while maintaining acceptable performance at scale.

2 Literature Map

[Placeholder: literature map to be inserted.]

3 Research Gap

Prior work shows that runtime and path-aware controls can detect multi-step policy violations, but existing evaluations mainly emphasize policy design and detection effectiveness in limited settings. Less evidence exists on how path-aware governance performs under high event volumes, longer workflows, and increasing agent concurrency, or whether its safety benefits justify the additional system overhead. This project addresses that gap by comparing no governance, lightweight per-action governance, and path-aware governance using detection accuracy, latency, throughput, and resource utilization.

4 Proposed Contributions: Novelty and Technical Depth

We are not claiming path-aware governance itself is new. Kaptein et al. (2026) lay out the theory but present no working system or experiments. Their own separate implementation admits that its shared memory across agents can go stale under concurrent load and “may miss violations in high-concurrency edge cases,” but they never measure how often that actually happens. VIGIL (2026) shows this kind of detection can work well, over 95% recall and under 10% false positives, but only for one agent at a time, with no shared memory between agents. Agent Behavioral Contracts

(2026) measures real overhead, under 10ms per action, also for a single agent. So the pieces exist separately: one agent’s detection accuracy is proven, one agent’s overhead is proven. Nobody has tested either one under real concurrency, many agents sharing memory at once, which is exactly the setup Kaptein et al. describe but never build.

Here is why that setup breaks in a specific way. To catch a violation that spans two agents, the system needs a shared memory of what each agent has already done. When that memory updates, it takes a small amount of time to reach every part of the system. With only a few agents acting occasionally, that tiny delay almost never matters. As concurrency rises, that window gets hit constantly, so a check can run against outdated memory and let a real violation through, not because the rule was wrong, but because the timing was bad. We build all three versions (no governance, per-action only, and path-aware) on the same test workflows and measure how often this actually happens, and what it costs in speed to prevent it, as the number of simultaneous agents increases. That measurement does not exist yet anywhere in the literature.

Getting that measurement right takes real work: keeping shared memory correct under concurrent writes is a real systems problem, the test workflows need cases that look like violations but are not so the results mean something, and we have to make sure that when things slow down, it is because of the governance check and not just because more agents are running at the same time.

5 Tasks to Be Done and Ownership

The implementation splits into five workstreams, each owned through integration.

- **Workflow and data generation:** build the synthetic e-commerce environment and roughly 100 to 200 labeled workflows spanning safe, single-action violation, multi-step violation, and near-miss legitimate categories.
- **Streaming pipeline:** set up Kafka topics, producers and consumers, partitioning by agent identity, and event replay at controlled concurrency levels.
- **Stateless governance:** implement the per-action baseline, evaluating each proposed action independently of history.
- **Path-aware governance:** implement the stateful engine, maintaining per-agent execution history and shared cross-agent memory to detect sequence-dependent violations.
- **Evaluation and monitoring:** build the benchmark harness that sweeps concurrency levels, logs results, and produces the accuracy and performance metrics, comparison graphs, and dashboard used in the final report.

Each workstream feeds its results into the integration and analysis weeks that follow.

6 Deliverables

- Governance middleware with three modes: no-governance, per-action, path-aware
- Kafka pipeline: agent-actions and governance-decisions topics, stateful stream processor with per-agent state store
- 100–200 synthetic workflows (safe, single-step violation, multi-step violation, near-miss legitimate)
- Benchmark harness collecting latency, throughput, and resource metrics via a load generator sweeping concurrency (10 to 100+)
- Benchmark report covering safety and performance results
- Demonstration: a short walkthrough of workflows run through all three modes
- Final report and presentation

7 Timeline (14 Weeks)

Weeks	Milestone
1–2	Literature verification finalized; architecture locked; dataset schema and violation taxonomy defined
3–4	Mocked tools and simulated agents running; ~50 workflows drafted (toward deliverable 3)
5–6	Middleware built; no-governance and per-action modes working end-to-end, no Kafka yet (deliverable 1, partial)
7–8	Kafka pipeline stood up; stateful stream processor with state store; path-aware mode integrated (deliverables 1 and 2 complete)
9	Buffer: resolve pipeline/middleware integration issues; finish dataset to full 100–200 workflows (deliverable 3 complete)
10–11	Benchmark harness built and run across all three modes and concurrency levels (deliverable 4 complete); dashboard finalized
12	Buffer: re-run experiments to catch anomalies; draft benchmark report (deliverable 5); select and rehearse demo workflows (deliverable 6)
13	Finalize benchmark report; rehearse live demo end-to-end at least twice; record fallback video in case of live failure
14	Report and presentation polish, final rehearsal, submission (deliverable 7)